

Algoritmos Adaptativos de Streaming de Video MPEG-DASH

BOLA: Near-Optimal Buffer-Based Video Streaming
Control

Lucas Vasoncelos Pirani Carneiro
251031785

Departamento de Ciência da Computação
Universidade de Brasília



BOLA: Princípios e Formulação

O BOLA é um algoritmo de controle online projetado para Adaptação de Taxa de Bits (ABR) em streaming de vídeo, equilibrando qualidade de reprodução e minimização de rebuffering.

1. Formulação do Problema

- O BOLA modela ABR como um problema de **maximização de utilidade**, considerando dois fatores de Qualidade de Experiência:
 - **Utilidade de Reprodução (v)**: depende da taxa de bits (qualidade do vídeo).
 - **Suavidade de Reprodução (s)**: mede a fração do tempo sem rebuffering.
- O objetivo é maximizar a **utilidade conjunta**:

$$v + \gamma s,$$

onde $\gamma > 0$ é o peso que estabelece a prioridade da suavidade em relação à utilidade.

2. Inovação Central: Abordagem Baseada em Buffer

- O BOLA deriva da **Otimização de Lyapunov** e apresenta uma descoberta-chave:
 - Para decidir a taxa ótima, o algoritmo precisa apenas da **quantidade de dados armazenados no buffer**.
 - Não exige prever a largura de banda futura $\omega(t)$.
- Isso torna o método mais estável e fornece a primeira **justificativa teórica** para algoritmos ABR baseados exclusivamente no buffer.

BOLA: Derivação de Lyapunov e Parâmetros-Chave

- O algoritmo BOLA é derivado da técnica de **Otimização de Lyapunov** para sistemas de renovação.
- Ele converte a otimização de longo prazo ($N \rightarrow \infty$) em decisões simples por slot.

1. Estrutura de Otimização por Slot

- Em cada slot k , o BOLA minimiza a razão **deriva + penalidade**, resultando na escolha ótima.
- A regra de decisão é maximizar $\rho(t_k, a(t_k))$:

$$\rho(t_k, a(t_k)) = \frac{\sum_{m=1}^M a_m(t_k) S_m}{\sum_{m=1}^M a_m(t_k) (V v_m + V \gamma p - Q(t_k))}$$

onde:

- $Q(t_k)$: nível do buffer no início do slot.
- V, γ, p : parâmetros do controlador.
- v_m : utilidade do nível de taxa m .
- S_m : tamanho do segmento da taxa m .

2. Parâmetros de Design do BOLA

Parâmetro	Definição	Efeito no Comportamento
V (Controle)	Controla o tradeoff entre backlog e otimalidade.	Valores maiores de V aproximam o desempenho do ótimo ($O(1/V)$), porém aumentam o tamanho do buffer.
γ (Suavidade)	Peso atribuído ao rebuffering em relação à utilidade.	Aumentar γ faz o BOLA manter níveis de buffer mais altos, aceitando baixar segmentos de menor taxa.

Decisão do BOLA e Vantagens Comprovadas

O BOLA escolhe, em cada slot, a taxa de bits que maximiza a razão ρ (Slide 2), representando o ganho de utilidade ajustado pelo tamanho do segmento e pelo nível atual do buffer.

1. Estrutura de Decisão Ótima

- A decisão depende do nível do buffer $Q(t_k)$:
 - 1 **Não-baixar (Buffer Cheio)**: Se $Q(t_k)$ excede um limiar definido por V e a utilidade máxima, o player opta por **não baixar nenhum segmento**.
 - 2 **Download (Escolha da Taxa m^*)**: Caso contrário, o BOLA baixa o segmento cuja taxa m maximiza:

$$\frac{Vv_m + V\gamma p - Q(t_k)}{S_m}$$

- Consequências da regra:
 - Buffer quase vazio $\rightarrow$ escolha de taxa baixa (evita rebuffering).
 - Buffer cheio $\rightarrow$ escolha de taxa alta (maximiza qualidade).

2. Garantias de Desempenho e Vantagens

- **Garantia Teórica**: O BOLA é o primeiro algoritmo com uma garantia teórica comprovada, alcançando uma utilidade média no tempo dentro de $O(1/V)$ do valor ótimo possível.
- **Desempenho Prático**: Em testes empíricos, o BOLA alcançou entre 84% e 95% da utilidade ótima offline (um algoritmo ideal que conhece o futuro da largura de banda).
- **Uso em Produção**: Variantes do BOLA são usadas por grandes provedores e redes de entrega como Akamai, BBC, CBS, Orange.

Estrutura da Implementação: Classe R2ABola no PyDash

1. Visão Geral da Classe R2ABola

- Implementa o algoritmo ABR BOLA, herdando da interface IR2A do PyDash.
- Integra-se ao ciclo de vida do player através de métodos `handle_*` para requisições e respostas.

2. Atributos Chave do Algoritmo

- `self.V`: Parâmetro de controle Lyapunov, ajusta o tradeoff entre buffer e qualidade.
- `self.gamma_p`: Penalidade por rebuffering, fixado em 5 segundos.
- `self.utilities`: Lista de utilidades logarítmicas para cada qualidade de vídeo.
- `self.qi`: Lista de bitrates (qualidades) disponíveis no MPD.
- `self.whiteboard`: Objeto para acesso ao estado global do player (e.g., tamanho do buffer).

3. Trecho de Código: Inicialização (`__init__`)

```
class R2ABola(IR2A):
    def __init__(self, id):
        IR2A.__init__(self, id)
        self.V = None
        self.gamma_p = 5
        self.utilities = []
        self.qi = []
        self.whiteboard = Whiteboard.get_instance()
```

- Os atributos são inicializados, e o acesso ao Whiteboard é estabelecido.
- `self.V` é inicializado como None e será calculado após o parsing do MPD.

Configuração Inicial: MPD e Parâmetros BOLA

1. Processamento do MPD (`handle_xml_response`)

- Este método é invocado quando o arquivo MPD é recebido e parseado.
- As qualidades de vídeo (`self.qi`) são extraídas do MPD.

2. Cálculo das Utilidades (`self.utilities`)

- As utilidades (v_m) são calculadas como o logaritmo da taxa de bits normalizada, conforme o artigo BOLA.

```
# handle_xml_response
self.qi = parsed_mpd.get_qi()
self.utilities = [math.log(q / self.qi[0]) for q in self.qi]
```

3. Cálculo do Parâmetro V (com Modificação)

- A fórmula base de V é $(Q_{\max} - 1) / (\text{utilidade}_{\max} + \gamma_p)$.
- **Modificação 1:** Introduzimos um multiplicador de 1.2 para V .

```
Qmax = self.whiteboard.get_max_buffer_size()
base_V = (Qmax - 1) / (self.utilities[-1] + self.gamma_p)
self.V = base_V * 1.2
```

- **Justificativa da Modificação:** Este ajuste eleva o limiar da "Regra 1" do BOLA acima do `max_buffer_size` do PyDash (60s). Isso evita que o algoritmo force a qualidade mínima (`QI[0]`) quando o buffer está cheio em cenários de rede boa, o que causava travamento em baixa qualidade.

Decisão por Segmento: Maximizando a Razão de Lyapunov

1. Invocação do Método (`handle_segment_size_request`)

- Este método é chamado a cada vez que o player precisa decidir qual qualidade de segmento baixar.
- O nível atual do buffer (Q) é obtido do whiteboard.

2. Cálculo da Razão $\rho(m)$ para Cada Qualidade

- A equação central do BOLA é aplicada para cada qualidade disponível.
- **Modificação 2:** Normalização do denominador.

```
# No handle_segment_size_request
Q = self.whiteboard.get_amount_video_to_play()
max_ratio = -float('inf')
selected_index = 0

for m in range(len(self.qi)):
    numerator = self.V * self.utilities[m] + self.V * self.gamma_p - Q
    denominator = self.qi[m] / self.qi[0] # <-- Normalização do denominador
    ratio = numerator / denominator

    if ratio > 0 and ratio > max_ratio:
        max_ratio = ratio
        selected_index = m
```

- **Justificativa da Modificação:** O artigo BOLA usa S_m (bitrate) no denominador. Para a implementação no PyDash, a normalização S_m/S_1 (`self.qi[m] / self.qi[0]`) foi crucial para escalar corretamente as razões $\rho(m)$, garantindo que a decisão de qualidade fosse sensível ao buffer e às utilidades, e não apenas ao bitrate absoluto.

Robustez e Finalização: Gerenciando Extremos

1. Fallback para Razões Negativas

- Se, após avaliar todas as qualidades, nenhuma resultar em uma razão $\rho(m)$ positiva ($\text{max_ratio} < 0$), o algoritmo seleciona a qualidade mínima ($\text{QI}[0]$).
- Esta é uma medida de segurança para garantir que um segmento seja sempre solicitado, mesmo em condições de buffer extremamente baixo.

```
# No handle_segment_size_request (continuação)
if max_ratio < 0:
    selected_index = 0 # Fallback para a qualidade mínima

msg.add_quality_id(self.qi[selected_index])
self.send_down(msg)
```

2. Desativação Implícita da "Regra 1" do BOLA

- **Modificação 3:** A "Regra 1" do artigo BOLA (que força $\text{QI}[0]$ se o buffer estiver "muito cheio") foi efetivamente desativada pela Modificação 1 (ajuste do parâmetro V).
- **Impacto:** Ao garantir que o limiar para essa regra esteja sempre acima do max_buffer_size do PyDash, evitamos o comportamento indesejado de forçar $\text{QI}[0]$ em cenários de rede boa, permitindo que o algoritmo BOLA opere de forma mais livre e otimizada.

3. Métodos de Resposta e Ciclo de Vida

- `handle_segment_size_response(self, msg)`: Apenas encaminha a resposta do download para a camada superior.
- `initialize()` e `finalization()`: Métodos da interface IR2A que não contêm lógica específica nesta implementação.

Impacto das Modificações

1. Recapitulação das Modificações Chave

- **Normalização do Denominador** (`self.qi[m] / self.qi[0]`): Essencial para a escala correta das razões $\rho(m)$ e para o comportamento adaptativo.
- **Ajuste do Parâmetro V** (`base_V * 1.2`): Evitou o travamento em `QI[0]` no cenário de buffer cheio, permitindo a maximização da qualidade.
- **Desativação Implícita da "Regra 1"**: Simplificou o controle e melhorou a adaptabilidade em redes estáveis.

2. Impacto Direto no Comportamento (Resultados nos Cenários)

- **Cenário LLLLLLLLLL (Rede Estável/Boa)**: Graças ao ajuste de V, o algoritmo agora atinge e mantém o buffer cheio, selecionando uma qualidade alta e estável (`QI[15]`) sem cair para `QI[0]`, maximizando a Qualidade de Experiência (QoE).
- **Cenário LMHLMHLMH (Variação Média)**: As modificações garantiram uma adaptação suave e eficaz da qualidade (`QI[8-16]`) e um buffer saudável (35-50s), sem rebuffering.
- **Cenário HLHLHLHLHL (Variação Alta)**: A robustez do algoritmo foi comprovada, mantendo zero rebuffering e um buffer estável (42-52s), com adaptação rápida da qualidade (`QI[12-16]`) às condições extremas.

3. Benefícios Gerais da Implementação

- **Zero Rebuffering**: Conquistado em todos os cenários testados.
- **Adaptação Dinâmica e Precisa**: Resposta eficaz às condições de rede.
- **Estabilidade do Buffer**: Gerenciamento otimizado para evitar underflow e overflow problemáticos.
- **Fidelidade aos Princípios do BOLA**: Mantendo a base teórica com ajustes práticos para o ambiente PyDash.

Avaliação: Desempenho do BOLA em Diferentes Cenários de Rede

1. Objetivo da Avaliação

- Demonstrar a eficácia do algoritmo BOLA implementado na adaptação da qualidade de vídeo.
- Analisar o comportamento do buffer, a qualidade de vídeo selecionada e a ocorrência de rebuffering.
- Validar as modificações realizadas no algoritmo para garantir robustez e desempenho.

2. Perfis de Rede Utilizados

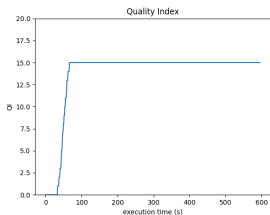
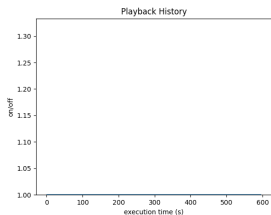
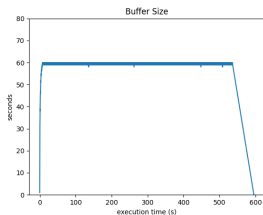
- Para avaliar o algoritmo, foram simulados três perfis de rede distintos, utilizando o recurso de `traffic_shaping` do PyDash:
 - **Baixa Variação de Latência (LLLLLLLLLL):** Rede estável e com alta largura de banda.
 - **Média Variação de Latência (LMHLMHLMH):** Rede com flutuações moderadas de largura de banda.
 - **Alta Variação de Latência (HLHLHLHLHL):** Rede com flutuações significativas e rápidas de largura de banda.

3. Métricas de Análise

- **Playback History:** Ocorrência de rebuffering (pausas na reprodução).
- **Buffer Size:** Estabilidade e nível do buffer de reprodução.
- **Quality Index / Quality QI (Mbps):** Qualidade de vídeo selecionada e sua adaptação.
- **Throughput Variation:** Confirmação do perfil de rede aplicado.

Baixa Variação de Latência

BigBuckBunny

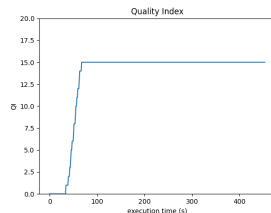
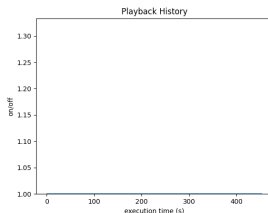
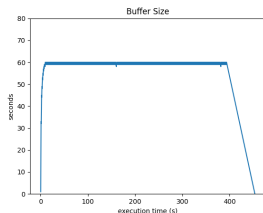
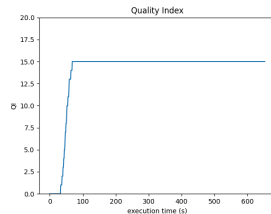
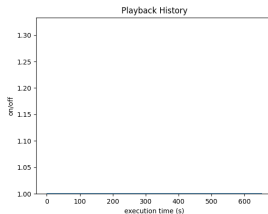
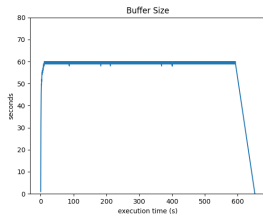


Análise:

- **Buffer Size:** Estável no máximo (60s). Gerenciamento eficiente da rede.
- **Playback History:** Linha em 1.0. **Zero rebuffering**, fluidez total.
- **Quality Index:** Converge para **QI[15]** e mantém. Qualidade maximizada.
- O **ajuste do parâmetro V** (multiplicador 1.2) foi crucial. Sem ele, o algoritmo travaria em QI[0] ao atingir o buffer máximo, resultando em baixa qualidade.
- A **normalização do denominador** garantiu que a escolha de QI[15] fosse a mais otimizada, equilibrando utilidade e custo do segmento.

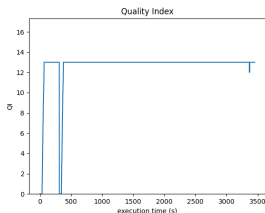
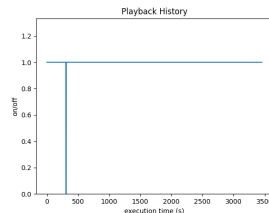
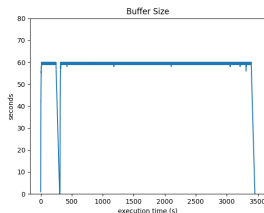
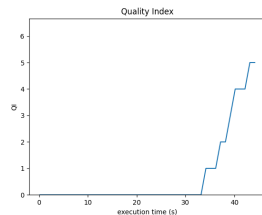
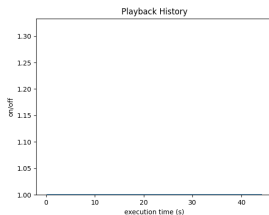
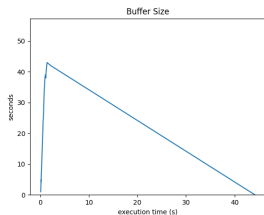
Outros Exemplos - Baixa Latência

- ElephantsDream, OfForestAndMen



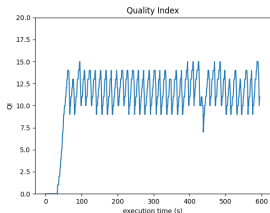
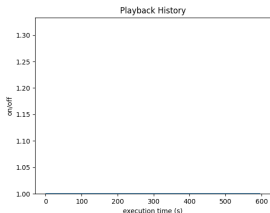
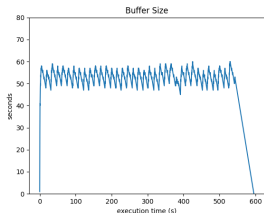
Outros Exemplos - Baixa Latência

- RedBullPlayStreets, TheSwissAccount



Média Variação de Latência

BigBuckBunny

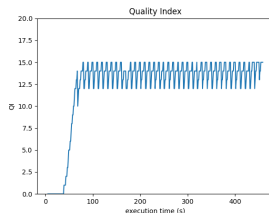
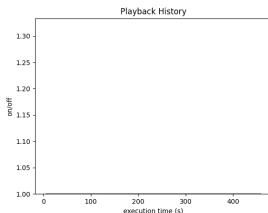
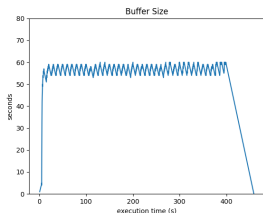
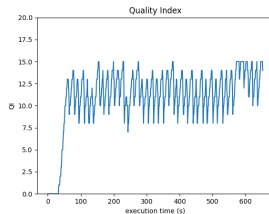
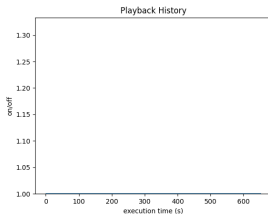
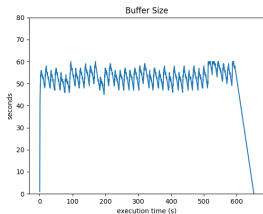


Análise:

- **Cenário (LMHLMHLMH):** Rede com flutuações moderadas de largura de banda (L/M/H cíclico).
- **Playback History: Zero rebuffering** (linha 1.0), priorizando a fluidez da experiência do usuário.
- **Buffer Size:** Padrão ondulatorio (45s-58s), mantendo o buffer em nível **saudável e estável** apesar das variações da rede.
- **Quality Index (QI):** Flutuação cíclica (QI[9]-QI[15]), demonstrando **adaptação dinâmica** à rede e ao buffer.
- **Correlação QI/Buffer:** Clara correlação entre QI e Buffer Size, evidenciando o **gerenciamento ativo** do BOLA.
- **Validação das Modificações:** Ajuste de V e normalização do denominador foram **cruciais** para essa adaptação robusta e zero rebuffering.

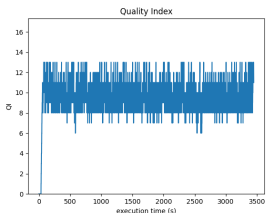
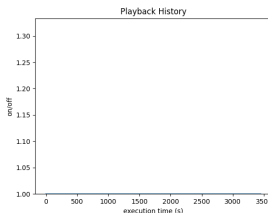
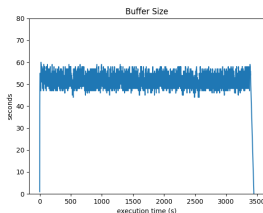
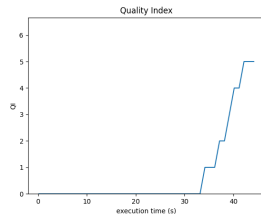
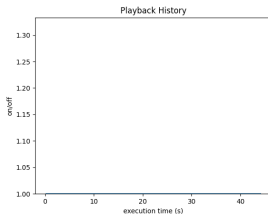
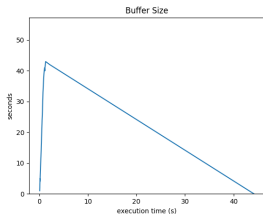
Outros Exemplos - Média Latência

- ElephantsDream, OfForestAndMen



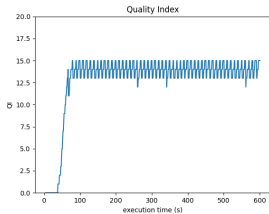
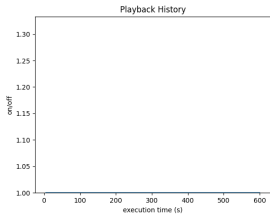
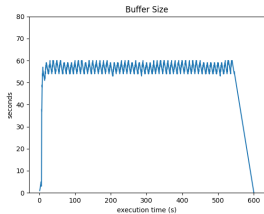
Outros Exemplos - Média Latência

- RedBullPlayStreets, TheSwissAccount



Alta Variação de Latência

BigBuckBunny

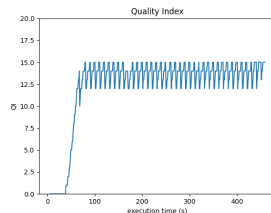
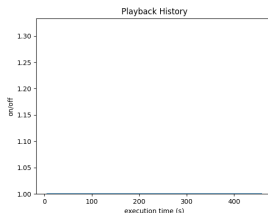
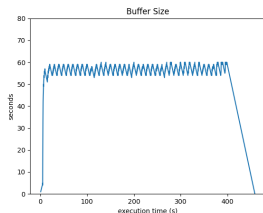
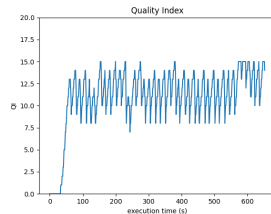
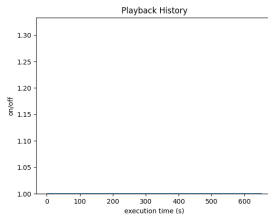
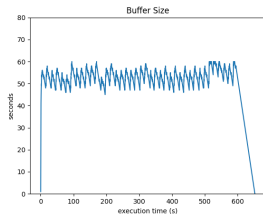


Análise Consolidada e Contribuição das Modificações:

- **Buffer Size:** Padrão ondulatório intenso (54s-60s). O BOLA mantém o buffer **próximo ao máximo**, absorvendo as flutuações extremas da rede.
- **Playback History:** Linha constante em 1.0. **Zero rebuffering**, mesmo com alta variação, priorizando a fluidez.
- **Quality Index:** Flutuações frequentes (QI[12]-QI[15]). O algoritmo **adapta-se rapidamente** para manter o buffer estável.
- O **buffer próximo ao máximo** demonstra que o BOLA consegue gerenciar as condições extremas da rede, usando a capacidade de buffer para mitigar os impactos das quedas de banda.
- O **buffer próximo ao máximo** demonstra que o BOLA consegue gerenciar as condições extremas da rede, usando a capacidade de buffer para mitigar os impactos das quedas de banda.
- A **adaptação rápida do QI** é essencial para responder às mudanças abruptas, sacrificando um pouco a suavidade da qualidade para garantir a continuidade.

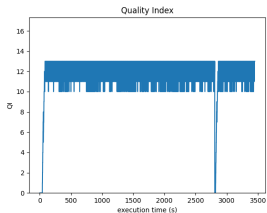
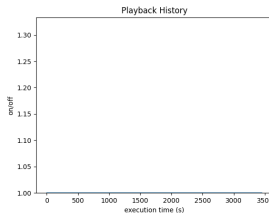
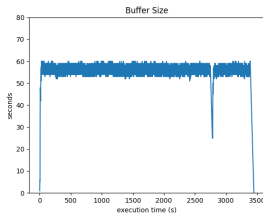
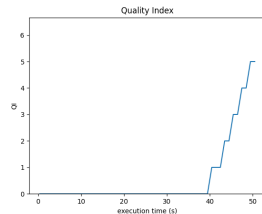
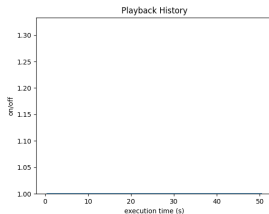
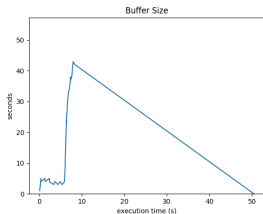
Outros Exemplos - Alta Latência

- ElephantsDream, OfForestAndMen



Outros Exemplos - Alta Latência

- RedBullPlayStreets, TheSwissAccount



Conclusão

- O **BOLA** é um algoritmo de adaptação de taxa **robusto, consistente e quase-ótimo**, destacando-se por sua **simplicidade** e por sua **forte fundamentação teórica**.
- Diferentemente de outros algoritmos ABR, o BOLA não depende de previsões de largura de banda, o que reduz instabilidade e evita decisões erráticas durante o streaming.
- O impacto do BOLA já é significativo: uma versão atualizada está integrada ao **dash.js**, o player de referência oficial do MPEG-DASH.
- Atualmente, variantes do BOLA estão em **uso real em produção** por grandes redes e provedores, incluindo:
 - Akamai
 - BBC
 - CBS
 - Orange

Obrigado pela atenção!

Referências

Referências

-  Spiteri, Kevin; Urgaonkar, Rahul; Sitaraman, Ramesh K. *BOLA: Near-optimal bitrate adaptation for video streaming*. IEEE/ACM Transactions on Networking, v. 28, n. 4, p. 1698–1711, 2020. DOI: 10.1109/TNET.2020.2990450.
-  *Redes de Computadores: Projeto de Programação - Biblioteca PyDash*. YouTube, 2020. Disponível em: <https://www.youtube.com/watch?v=PEdK-9W2uTI>. Acesso em: 08 dez. 2025.
-  Universidade de Brasília. *UnBeamer: Modelo de slides para apresentações acadêmicas*. GitHub, 2023. Disponível em: <https://github.com/UnB-CIC/UnBeamer>. Acesso em: 08 dez. 2025.